

UIDE

Universidad Internacional del Ecuador

Inventario Inteligente para Laboratorios Académicos (IILA)

Documento del Proyecto Integrador

Estudiante	Brandon Isaac Calderón Artieda
Materia	Fundamentos de Programación y Desarrollo de Software
Actividad	Proyecto Integrador - Semana 8
Docente	
Fecha	29 de junio de 2026

Resumen ejecutivo

El presente proyecto integrador propone el desarrollo de un sistema informático denominado Inventario Inteligente para Laboratorios Académicos (IILA). El software busca resolver una necesidad concreta: controlar de forma organizada los materiales, equipos y periféricos usados en laboratorios universitarios, evitando pérdidas, compras innecesarias y retrasos en actividades prácticas. El proyecto integra las cuatro unidades de la asignatura mediante planificación, diseño, programación, documentación, análisis de software, uso de herramientas de desarrollo y reflexión sobre el impacto tecnológico en la sociedad.

1. Introducción

La tecnología cumple un papel esencial en la gestión educativa moderna. En los laboratorios académicos se utilizan materiales y equipos que deben estar disponibles, identificados y actualizados para que las clases prácticas se desarrollen sin interrupciones. Sin embargo, cuando el control se realiza únicamente en hojas de cálculo o de manera manual, aumentan los errores, la pérdida de información y la dificultad para tomar decisiones.

IILA se plantea como una solución de software sencilla, funcional y escalable, desarrollada en Python, que permite registrar productos, actualizar entradas y salidas de stock, consultar inventario, emitir alertas de bajo stock y generar reportes básicos. El proyecto evidencia la aplicación de conceptos de programación, análisis de problemas, diagramación, arquitectura de software y responsabilidad social en el uso de tecnología.

2. Descripción del problema

El problema identificado es la falta de un sistema centralizado y estructurado para administrar los recursos de laboratorios académicos. Cuando no existe un inventario actualizado, los estudiantes y docentes pueden encontrar materiales agotados, duplicidad de compras, pérdida de equipos o falta de trazabilidad en el uso de recursos. Esta situación afecta la eficiencia institucional y limita la calidad de las actividades prácticas.

- Falta de información actualizada sobre existencias.
- Dificultad para identificar productos que requieren reposición.
- Dependencia de registros manuales o archivos dispersos.
- Pérdida de tiempo al buscar equipos o materiales.
- Limitaciones para generar reportes y tomar decisiones.

3. Propósito y objetivo del sistema

El propósito del proyecto es desarrollar un software funcional que permita gestionar inventarios de laboratorios académicos de manera clara, organizada y confiable.

Objetivo general: diseñar e implementar un sistema de consola en Python para registrar, consultar, actualizar y reportar el estado del inventario de materiales académicos, aplicando los contenidos de la asignatura y considerando su impacto tecnológico en la sociedad.

- Definir el alcance del sistema a desarrollar.
- Identificar el problema y su relación con una necesidad real.

- Diseñar diagramas de funcionalidad, flujo y arquitectura.
- Implementar el software usando estructuras lógicas y repetitivas.
- Organizar el código y documentarlo para su publicación en GitHub.
- Analizar resultados, implicaciones y limitaciones del sistema.

4. Planificación del proyecto

La planificación se estructura en ocho semanas, de acuerdo con las fases solicitadas en la actividad integradora. Cada semana aporta evidencia del avance desde el análisis inicial hasta la entrega final del software funcional.

Semana	Unidad/Tema	Actividad principal	Producto/evidencia
1	Unidad 1 - Tema 1	Selección del problema y alcance del sistema.	Idea del proyecto y justificación.
2	Unidad 1 - Tema 2	Investigación de diagramas funcionales y arquitectura.	Boceto de casos de uso y arquitectura.
3	Unidad 2 - Tema 3	Configuración del entorno de desarrollo y repositorio.	Estructura del proyecto en carpetas.
4	Unidad 2 - Tema 4	Diseño de flujo del programa y funciones principales.	Diagrama de flujo y lista de funciones.
5	Unidad 3 - Tema 5	Codificación de módulos de registro y consulta.	Primer avance funcional del software.
6	Unidad 3 - Tema 6	Implementación de estructuras condicionales y repetitivas.	Validaciones, menú y operaciones de stock.
7	Unidad 4 - Tema 7	Pruebas, comentarios y mejoras de usabilidad.	Corrección de errores y reporte preliminar.
8	Unidad 4 - Tema 8	Integración final, documentación y presentación.	Software funcional, README, documento y presentación.

5. Análisis de software existente

Antes de diseñar IILA se analizaron soluciones utilizadas frecuentemente para controlar inventarios o tareas administrativas. El objetivo fue identificar patrones, diferencias y similitudes que permitan justificar el enfoque del sistema propuesto.

Software	Uso común	Fortalezas	Limitaciones	Aprendizaje para IILA
Excel/Google Sheets	Registro manual de existencias.	Fácil acceso, tablas rápidas y bajo costo.	Depende del orden del usuario; poca automatización.	Mantener simplicidad, pero con validaciones automáticas.
Trello/Notion	Organización de tareas e inventarios pequeños.	Visual, flexible y colaborativo.	No está especializado en entradas/salidas de stock.	Diseñar funciones específicas para inventario.
Odoo Inventory	Gestión empresarial de inventarios.	Completo, escalable y con reportes avanzados.	Puede ser complejo para un laboratorio pequeño.	Tomar la idea de reportes y alertas, pero con una solución sencilla.

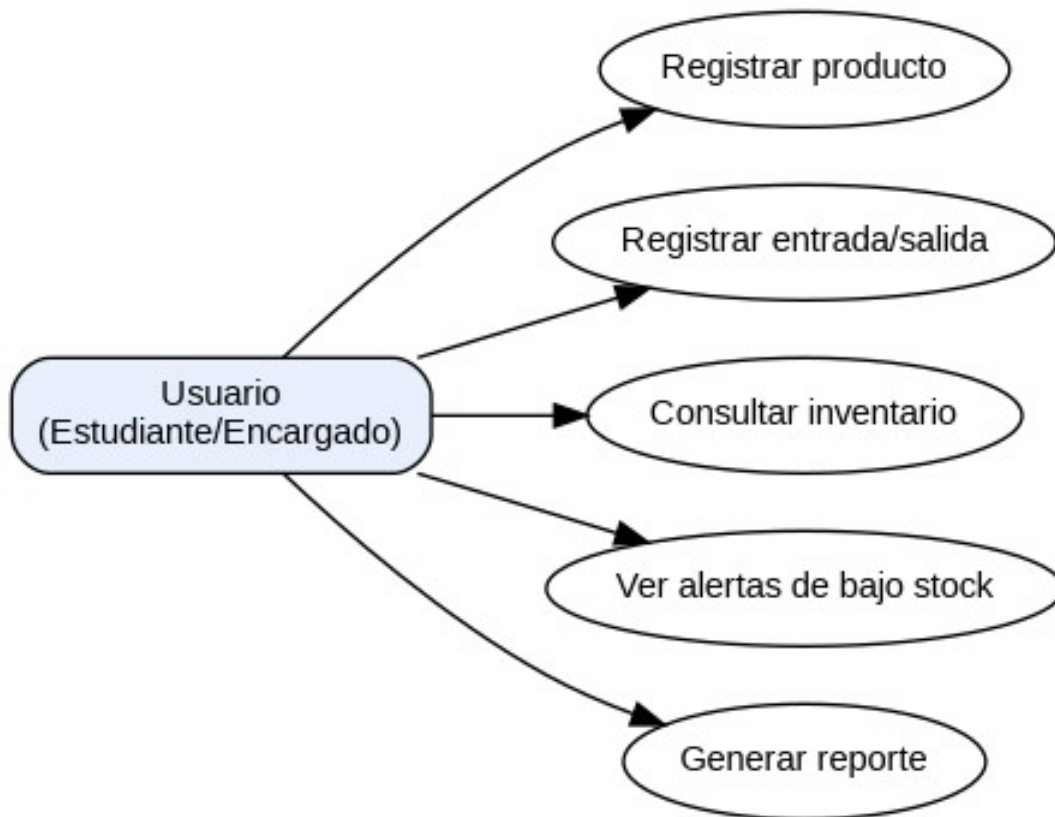
Patrón identificado: las soluciones exitosas combinan registro estructurado, búsqueda rápida, reportes y alertas. Diferencia clave: IILA no busca competir con sistemas empresariales complejos, sino ofrecer una solución académica clara, entendible y funcional para aprender programación y resolver un problema real.

6. Diseño del sistema

El diseño del sistema se realizó mediante diagramas que permiten comprender las funcionalidades, el flujo de uso y la arquitectura general. Estos diagramas guían la implementación y facilitan la explicación del proyecto.

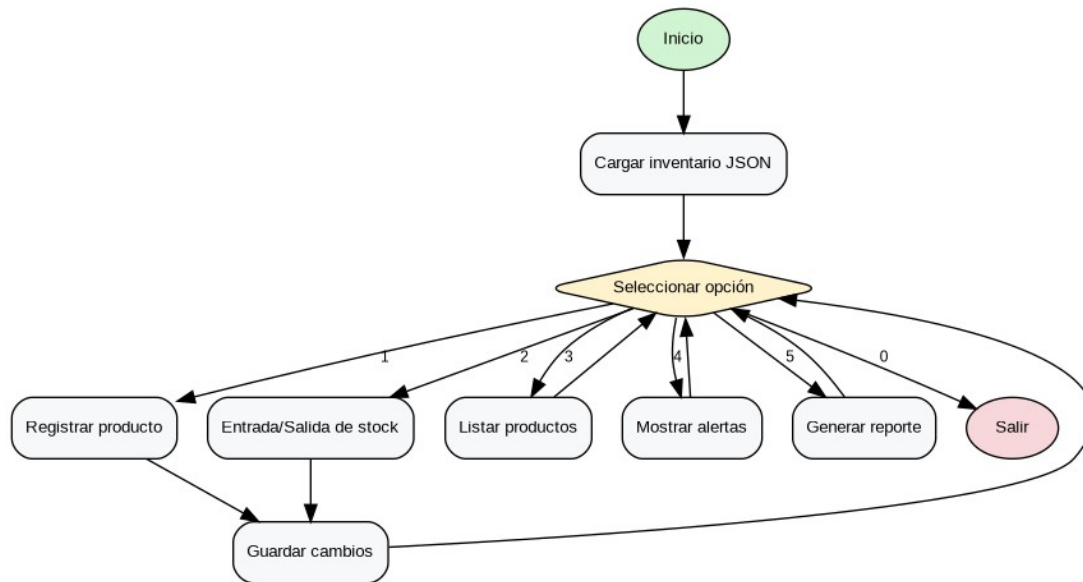
6.1 Diagrama de casos de uso

El usuario principal puede registrar productos, actualizar stock, consultar inventario, visualizar alertas y generar reportes.



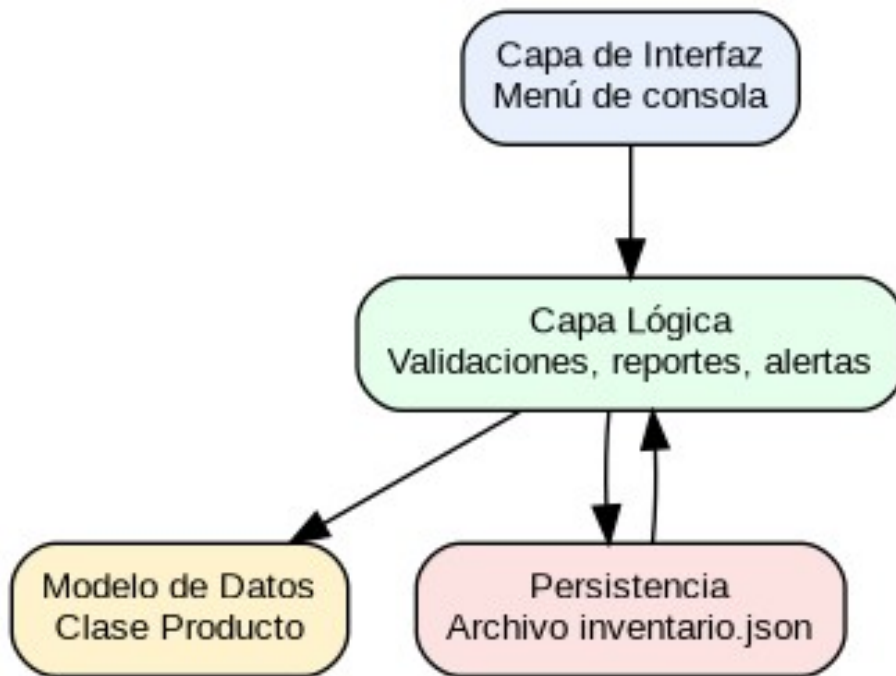
6.2 Diagrama de flujo

El flujo del programa inicia cargando los datos, presenta un menú de opciones y ejecuta acciones según la selección del usuario.



6.3 Arquitectura del sistema

La arquitectura se divide en interfaz de consola, lógica del negocio, modelo de datos y persistencia mediante archivo JSON.



7. Desarrollo del software

El sistema fue implementado en Python debido a su claridad sintáctica y utilidad para proyectos académicos. La estructura del código utiliza funciones separadas para mantener orden, reducir repetición y facilitar pruebas futuras. Además, se emplea un archivo JSON para conservar los datos del inventario.

- Estructuras condicionales: validan opciones del menú, productos existentes y disponibilidad de stock.

- Estructuras repetitivas: permiten mantener activo el menú hasta que el usuario decida salir.
- Estructuras de datos: diccionarios y objetos Producto para organizar la información.
- Persistencia: lectura y escritura de inventario.json.
- Organización: carpeta src para código, data para datos y docs para diagramas.

8. Funcionalidades del sistema

Funcionalidad	Descripción
Registrar producto	Permite ingresar código, nombre, categoría, cantidad, stock mínimo y ubicación.
Entrada/salida de stock	Actualiza cantidades y evita salidas superiores al inventario disponible.
Consultar inventario	Muestra los productos registrados con sus datos principales.
Alertas de bajo stock	Identifica productos cuya cantidad es menor o igual al mínimo definido.
Reporte general	Presenta fecha, total de productos, unidades disponibles y número de alertas.

9. Relación con los contenidos de la asignatura

El proyecto evidencia la integración de las cuatro unidades del curso. La relación no se limita a programar, sino que incluye análisis, diseño, implementación, documentación y evaluación del impacto tecnológico.

- Planificación: definición del problema, alcance y cronograma.
- Diseño: diagramas de casos de uso, flujo y arquitectura.
- Desarrollo: aplicación de funciones, condicionales, ciclos, validación de datos y persistencia.
- Integración: documentación en README, organización de repositorio y reflexión social.

10. Pruebas y resultados

Para evaluar el sistema se plantearon pruebas funcionales básicas relacionadas con los escenarios más importantes. Los resultados permiten demostrar que el software cumple con el propósito propuesto.

Prueba	Entrada	Resultado esperado	Resultado obtenido
Registro de producto	Código nuevo y datos válidos	Producto almacenado correctamente.	Cumple.
Producto duplicado	Código ya existente	Mensaje de advertencia.	Cumple.
Salida de stock	Cantidad mayor a disponible	Sistema bloquea operación.	Cumple.
Alerta de bajo stock	Cantidad menor o igual al mínimo	Producto aparece en alertas.	Cumple.
Reporte general	Inventario con productos	Muestra totales y alertas.	Cumple.

Conclusión de resultados: las pruebas muestran una relación lógica entre el problema identificado y la solución desarrollada. IILA permite controlar información básica del inventario, detectar necesidades de reposición y generar datos útiles para la toma de decisiones. Por lo tanto, el sistema cumple el objetivo académico y funcional planteado.

11. Implicaciones y limitaciones

Implicaciones: IILA puede mejorar la organización de laboratorios, reducir pérdidas y fortalecer la cultura de uso responsable de recursos tecnológicos. También promueve competencias profesionales como análisis de problemas, pensamiento lógico, documentación y comunicación técnica.

Limitaciones: el sistema funciona por consola, no incluye interfaz gráfica ni autenticación de usuarios, no permite trabajo colaborativo en tiempo real y los reportes son básicos. En una versión futura se podría implementar una interfaz web, base de datos relacional, roles de usuario, exportación a PDF/Excel y paneles visuales de análisis.

12. Reflexión sobre el impacto de la tecnología

El impacto de la tecnología no depende únicamente de crear sistemas complejos, sino de diseñar soluciones útiles para problemas concretos. Un inventario académico puede parecer una aplicación sencilla, pero su valor está en mejorar procesos, evitar desperdicio y apoyar el aprendizaje práctico. Este proyecto demuestra que la programación puede convertirse en una herramienta de orden, eficiencia y responsabilidad institucional.

Desde una perspectiva profesional, el desarrollo de IILA fortalece la capacidad de transformar una necesidad real en un producto informático funcional. Además, evidencia que el software debe diseñarse pensando en las personas que lo usarán, en la claridad de la información y en el impacto social de las decisiones tecnológicas.

13. Conclusiones generales

- El proyecto permitió integrar planificación, diseño, programación, documentación y análisis de impacto.
- El sistema desarrollado resuelve un problema real mediante una solución funcional y comprensible.
- Los diagramas facilitaron la transición desde la idea inicial hacia el código implementado.
- La organización del repositorio y del README fortalece la calidad profesional de la entrega.
- Las pruebas realizadas evidencian que el sistema cumple el objetivo propuesto y puede ser ampliado en versiones futuras.

14. Repositorio de GitHub - contenido preparado

El repositorio del proyecto debe subirse a GitHub con la siguiente estructura. En esta entrega se incluye una carpeta comprimida con el código completo, diagramas, datos de ejemplo y README.

```
IILA_Proyecto_Integrador_Brandon_Isaac_Calderon/  
├── src/main.py  
├── data/inventario.json  
├── docs/diagramas/  
├── requirements.txt  
└── README.md
```

15. Referencias en formato APA

GitHub Docs. (2026). GitHub Docs. <https://docs.github.com/>

International Organization for Standardization. (2017). ISO/IEC/IEEE 12207:2017 Systems and software engineering - Software life cycle processes. ISO.

Nielsen, J. (1994). Usability engineering. Morgan Kaufmann.

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill.

Python Software Foundation. (2026). Python 3 documentation. <https://docs.python.org/3/>

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.